



Incorporate seagull optimization into ordinal optimization for solving the constrained binary simulation optimization problems

Shih-Cheng Horng¹ · Shieh-Shing Lin¹

Accepted: 5 October 2022 / Published online: 26 October 2022

© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2022

Abstract

Constrained binary simulation optimization problems (CBSOP) are optimization problems with binary variables and stochastic objective function subject to given constraints. Solving the CBSOP by conventional optimization algorithms becomes highly time-consuming when the problem size is increased. Although the ordinal optimization (OO) theory provides a reliable framework to solve CBSOP, the constraints still limit the efficiency and competitiveness of the OO theory. In this work, an approach incorporating binary seagull optimization into ordinal optimization, abbreviated as BSOO, is developed for solving the CBSOP in a reasonable time. The BSOO comprises three essential components: emulator, exploration, and exploitation. First of all, the regularized minimal-energy tensor product B-splines are regarded as an emulator to estimate the performance of a solution. Next, the binary seagull optimization algorithm is utilized to determine N exceptional solutions from the decision space. Finally, the reformed optimal computing budget allocation is employed to find an illustrious solution from the N exceptional solutions. To verify the proposed method, the BSOO is applied for finding the optimal layout of shortcuts for maximizing the capacity of the sorting conveyor system in a reasonable time. Experimental results of the BSOO are compared to five heuristic methods. The BSOO outperforms the five heuristic methods even after the latter took more than 30 times the CPU time that was consumed by BSOO upon completion. Test results reveal that the BSOO can be adopted in a real-time application of the sortation system.

Keywords Binary seagull optimization · Ordinal optimization · Tensor product B-splines · Optimal computing budget allocation · Sorting conveyor system · Shortcuts layout

✉ Shih-Cheng Horng
schong@cyut.edu.tw

Extended author information available on the last page of the article

List of symbols

$x = [x_1, \dots, x_K]^T$	A binary solution
$f(x)$	Performance metric of the system
$E[f(\mathbf{x})]$	Expectation of system's performance
$h_i(\mathbf{x})$	The i th constrained function
d_i	The i th service level
$\bar{f}(\mathbf{x})$	Sample mean
L	The number of replications
$f_{\ell}(\mathbf{x})$	The estimate of the ℓ th replication
η	Penalty weight
$F(\mathbf{x})$	Penalized objective function
$pe_i(\mathbf{x})$	Quadratic penalty function
L_a	The replications of the accurate estimate
$F_a(\mathbf{x})$	Penalized objective function using an accurate estimate
P	The number of basis functions
w_i	Coefficient of the B-spline
$\Phi_i(\mathbf{x})$	B-splines basis functions
$\mathbf{Q}$	n Coefficients vector
$B_d(\mathbf{x})$	Vector of n B-spline piecewise polynomial functions
$\Phi(\mathbf{x}_i)$	Mapping vector of the spline coefficients
$\mathbf{w}$	The spline coefficients vector
$\mathbf{H}$	The discretized matrix
ξ	The regularization term on gradient training data
δ	The regularization term penalizing the norm of the spline coefficients
A	Factor for controlling the movement behavior of the seagulls
B	Factor for balancing between exploration and exploitation
Ψ	Total number of seagulls
$t_{\max}$	The required iterations
$\mathbf{x}_i^t$	The position of the i th seagull at iteration t
$\mathbf{z}_i^t$	The search position of the i th seagull that does not collide with other seagulls at iteration t
$\mathbf{y}_i^t$	The search direction of the i th seagull toward the elite seagull at iteration t
$\mathbf{D}_i^t$	The distance between the i th seagull and the elite seagull at iteration t
$\mathbf{V}_i^t$	The speed of the i th seagull at iteration t
$\mathbf{x}^*$	The position of the best-so-far elite seagull
$A_{\min}$	The lower bound of A
$A_{\max}$	The upper bound of A
$B_{\min}$	The lower bound of B
$B_{\max}$	The upper bound of B
α_k	The profit of knapsack example
β_k	The weight of knapsack example
C_b	The allowable computing effort

N	Number of exceptional solutions
L_0	The basic replications
L_i	The replications allocated to the i th exceptional solution
Δ	An additional computing budget
s	A time-reducing parameter
$\mathbf{X} = [x_{ij}]_{I \times J}$	A solution matrix
x_{ij}	A 0–1 variable from start point i to endpoint j
λ	The arrival interval rate of the parcel
$E[f(\mathbf{X}, \lambda)]$	Throughput of the system
D	The required number of the shortcut
$\bar{f}(\mathbf{X}, \lambda)$	The sample mean of throughput
$F(\mathbf{X}, \lambda)$	A penalized objective function
Π	Number of randomly chosen samples
Ω	A representative subset
r	The rank of an illustrious solution in Ω

1 Introduction

Constrained binary simulation optimization problems (CBSOPs) are optimization problems with binary variables and stochastic objective function subject to given constraints [1]. Such problems occur in almost all application areas, such as portfolio optimization in finance, resource allocation of the grid computing system, traffic management and scheduling in transportation, lead optimization in pharmaceutical drug discovery, and layout optimization of the sortation conveyor system. Since the decision space often lacks a rich structure that can be used to identify the optimal solution, the CBSOP belongs to the complexity class of NP-hard problems [2, 3]. In addition, the constraints make the CBSOP difficult to determine an illustrious solution in a reasonable time by the classical optimization techniques.

One approach for solving CBSOP is continuous in nature. However, the obtained solution may be neither optimal nor feasible. Alternative approaches involve the cutting-plane and branch-and-cut. The cutting-plane approach solves the linear programming relaxation and adds linear constraints that prompt the solution toward integers [4]. The branch-and-cut approach combines branch-and-bound and cutting-plane methods. It partially builds a binary tree and iteratively cuts out the nodes that have a lower bound worse than the current upper bound [5]. However, local minima trapping and slow convergence rate are the disadvantages of these two methods.

Several meta-heuristic algorithms are used for resolving CBSOP, including genetic algorithm (GA) [6], ant colony optimization (ACO) [7], and clonal selection algorithm (CSA) [8]. However, these meta-heuristic methods are computationally intensive and may get stuck at a local optimum. Recently, swarm intelligence (SI) is a promising way to develop powerful solution methods for CBSOP. SI [9] is inspired by self-organizing biological processes and the behavior of swarms of living creatures such as ant colonies, bird flocks, bee colonies, and fish schools. Some examples of novel bio-inspired intelligence approaches include Aquila optimizer (AO) [10], horse herd optimization algorithm (HOA) [11], golden eagle optimizer (GEO)

[12], whale optimization algorithm (WOA) [13], equilibrium optimization algorithm (EOA) [14], and seagull optimization algorithm (SOA) [15, 16]. Since a population can spread in different regions of the decision space, convergence of SI approaches is much faster than the traditional approaches.

The CBSOP becomes more difficult due to the following three issues, (i) a large decision space, (ii) evaluation of a stochastic objective function is highly time-consuming, and (iii) all constraints should be satisfied. The ordinal optimization (OO) theory [17, 18] is developed to attack this difficulty by solving issues (i) to (iii) simultaneously. The OO theory aims to look for good enough solutions with high probability instead of finding the perfect optimum to avoid time-consuming calculations. With the aid of a rough estimate which gives a rough ranking of the solutions, the random choice of solutions would cover some good enough solutions. Since the rough estimate is much faster than the accurate estimate, the OO theory can enable an overall performance improvement of the considered problem. The OO theory has three basic procedures. Firstly, a large number of solutions are generated, and a crude evaluation is adopted to quickly estimate the objective value of each solution. Secondly, these solutions are sorted by the evaluated performances, and a certain number of excellent solutions are selected to estimate their performances using the accurate estimate. Finally, good enough solutions are chosen based on the results obtained by accurate estimate. There are many applications of the OO theory being used today to resolve the hard simulation optimization problems, such as the design of queuing communication systems [19], call center staffing optimization [20], pull-type manufacturing systems [21], and optimal size of manufacturing facilities [22].

The OO theory helps to diminish the number of structure options based on the approximate reliabilities. Although the OO theory can accelerate the screening process, the constraints drastically drag the efficiency. For reducing the computing time of CBSOP, an approach incorporating binary seagull optimization into ordinal optimization (BSOO) is proposed to determine an illustrious solution within an acceptable time. The BSOO includes three modules, which are emulator, exploration, and exploitation. Firstly, an emulator based on the regularized minimal-energy tensor product B-splines (RMTB) [23, 24] is regarded as a performance evaluation of a solution. Next, a binary seagull optimization algorithm (BSOA) is proposed to select N exceptional solutions from the decision space. Finally, a reformed optimal computing budget allocation (ROCBA) is adopted to look for an illustrious solution from the N exceptional solutions. These three modules dramatically decrease the overhead for solving CBSOP.

To verify the proposed method, the BSOO approach is applied to the optimal layout of shortcuts in the sorting conveyor system, which is modeled as a CBSOP. This CBSOP aims to seek the optimal layout of shortcuts for maximizing the capacity of the sorting conveyor system. This paper has two contributions. The *first* one is to develop a BSOO approach to seek an illustrious solution of a CBSOP within an acceptable period of time. Since most of the past researches did not utilize shortcuts in a sorting conveyor system, the *second* contribution is twofold: academic and practical perspective. From the academic perspective, the BSOO approach is developed to determine the optimal layout of shortcuts in the sorting conveyor system. From the practical perspective, this research aims to validate the BSOO approach for a practical case with 2 inbound conveyors, 10 outbound conveyors and a loop conveyor.

The rest of this paper progresses as follows. Section 2 introduces the BSOO approach to find an illustrious solution for a CBSOP. Section 3 presents the optimal layout of shortcuts in the sorting conveyor system, which is modeled as a CBSOP. The BSOO is then adopted to solve this CBSOP. Section 4 shows the cases studied, the results obtained, and the performed comparisons. The conclusion is summarized in the last section.

2 Incorporating binary seagull optimization into ordinal optimization

2.1 Constrained binary simulation optimization problems

The CBSOP is challenging in that the decision space often lacks a rich structure that can be used to identify the optimal solution. A typical CBSOP is represented as follows.

$$\max E[f(x)] \quad (1)$$

$$\text{subject to } h_i(x) = d_i, i = 1, \dots, I \quad (2)$$

$$x_k \in \{0, 1\}, k = 1, \dots, K \quad (3)$$

where $\mathbf{x} = [x_1, \dots, x_K]^T$ is a binary solution, x_k is a 0–1 variable, $f(\mathbf{x})$ is the performance metric of the system, $E[f(\mathbf{x})]$ is the expectation of system's performance, I denotes the number of constraints, and $h_i(\mathbf{x})$ and d_i represent the i th constrained function and service level, respectively. The goal of CBSOP is to find the binary solution (3) that will maximize the expectation of system's performance (1) while satisfying all the equality constraints (2).

A sufficiently large number of replications should be performed to produce accurate estimates of $E[f(\mathbf{x})]$. There is an alternative formula for estimating the value of $E[f(\mathbf{x})]$, which is the sample mean defined as follows.

$$\bar{f}(x) = \frac{1}{L} \sum_{\ell=1}^L f_{\ell}(x) \quad (4)$$

where L indicates the number of replications, and $f_{\ell}(\mathbf{x})$ depicts the estimate of the ℓ th replication. The value of $\bar{f}(\mathbf{x})$ has a better estimate of $E[f(\mathbf{x})]$ when the quantity of replications increases. The value of L must be sufficiently large when an accurate estimate about $E[f(\mathbf{x})]$ is needed.

Because the equality constraints are treated as soft constraints, the CBSOP can be transformed into an unconstrained one through penalty function methods [25]. The penalty function is used to relax all soft constraints and ensure that the constraints will be violated only if there exists no solution where all constraints are satisfied.

$$\max F(\mathbf{x}) = \bar{f}(\mathbf{x}) - \eta \times \sum_{i=1}^I pe_i(\mathbf{x}) \quad (5)$$

where η indicates a penalty weight, $F(\mathbf{x})$ indicates a penalized objective function, and $pe_i(\mathbf{x})$ denotes the following quadratic penalty function.

$$pe_i(\mathbf{x}) = \begin{cases} 0, & \text{if } h_i(\mathbf{x}) = d_i, \\ (h_i(\mathbf{x}) - d_i)^2, & \text{else,} \end{cases} \quad i = 1, \dots, I \quad (6)$$

A large penalty weight is adopted to assure all constraints are satisfied. Let L_a express the large enough of L , and the accurate estimate of (4) is referred to as $L = L_a$. For simplicity, we denote $F_a(\mathbf{x})$ as the penalized objective function of $\mathbf{x}$ computed using an accurate estimate.

The OO theory [17] clarifies that orders of solutions are probably preserved even if they are estimated by a rough model such as an emulator. Thus, an emulator based on the RMTB can be adopted to evaluate a solution more rapidly. Accordingly, the BSOA collocated with this emulator is adopted to select N exceptional solutions from decision space within a reasonable time.

2.2 Emulator

Many studies have been published regarding emulators in recent years, including the multivariate adaptive regression splines [26], support vector regression [27], extreme learning machines [28], polynomial chaos expansions [29], and regularized minimal-energy tensor product B-splines (RMTB) [23,24]. Among them, RMTB utilizes regularization and minimal energy to enhance accuracy for large unstructured datasets of low-dimensional problems. There are three advantages of RMTB: (i) scalability to the variability in parameters, (ii) fast prediction time, and (iii) unsusceptible to training failure. RMTB has been widely used in different applications, such as prediction, forecasting, curve fitting, classification, and function approximation [23]. The performance of RMTB has been compared to inverse distance weighting, radial basis functions, and kriging in terms of accuracy and efficiency [23]. When large datasets are available, the accuracy that be achieved with RMTB is expected to be higher than with the three regression methods. Accordingly, RMTB is the most appropriate alternative since a large set of data points is available in the application of the sorting conveyor system. Thus, an emulator according to the RMTB is adopted to assess the objective value of a solution.

The RMTB is composed of two layers as shown in Fig. 1. Arbitrary choose Π $\mathbf{x}$'s from decision space and calculate $F_a(\mathbf{x})$ by accurate estimate. We represent these Π training samples as $(\mathbf{x}_i, F_a(\mathbf{x}_i))$, where $\mathbf{x}_i$ and $F_a(\mathbf{x}_i)$ depict a solution and whose corresponding penalized objective value calculated by accurate estimate, respectively. The RMTB model can predict $F(\mathbf{x})$ by sums of multivariate B-splines.

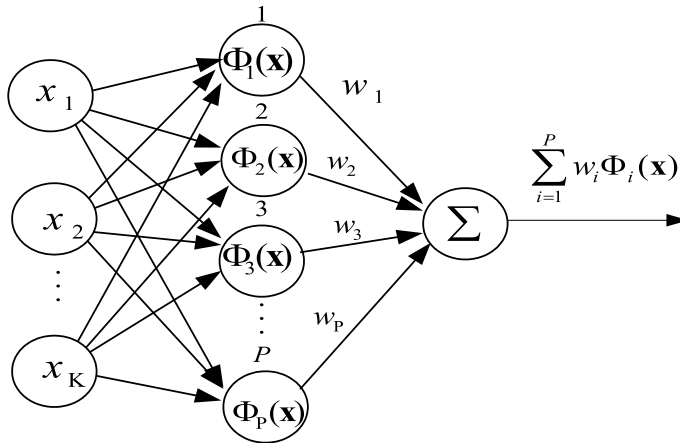


Fig. 1 Framework of an RMTB

$$F(x) = \sum_{i=1}^P w_i \Phi_i(x) \quad (7)$$

where P depicts the number of basis functions, w_i depicts the coefficient of the B-spline, and $\Phi_i(\mathbf{x})$ are B-spline basis functions as follows.

$$\Phi(x) = Q^T B_d(x) \quad (8)$$

where Q denotes an n coefficients vector, and $B_d(\mathbf{x})$ indicates a vector of n B-spline piecewise polynomial functions.

RMTB yields the coefficients w_i of the splines by solving an energy minimization problem such that the splines pass through the training points. The energy minimization problem contains three terms: energy term, regularization, and approximation error in fitting the training points [23].

$$\min_w \frac{1}{2} \left(w^T H w + \delta w^T w + \frac{1}{\xi} \sum_{i=1}^P (\Phi(x_i) w - F_a(x_i))^2 \right) \quad (9)$$

where $\Phi(x_i) = [\Phi_1(x_i), \dots, \Phi_P(x_i)]$ denotes the mapping vector of the spline coefficients to the i th predicted output, $w = [w_1, \dots, w_P]^T$ represents the spline coefficients vector, H denotes the discretized matrix for the continuous energy functional associated with the second-order derivatives of the spline curve, ξ denotes the regularization term on gradient training data, and δ denotes the regularization term penalizing the norm of the spline coefficients.

The RMTB is trained offline to greatly reduce the online computational burden. After termination of training RMTB, the model prediction of $F(\mathbf{x})$ can be obtained by simple mathematical calculations for a solution $\mathbf{x}$.

2.3 Exploration

2.3.1 Seagull optimization algorithm

With the aid of the RMTB emulator, we can apply the existing optimization methods to find N exceptional solutions from the decision space. Because the SOA improves the individuals as the population evolves across generations, it is best suited for specific requirements. SOA is a swarm-based meta-heuristic method that mimics the migration behaviors and attacking behaviors of a seagull in nature [15]. The SOA has many advantages including low-computational cost, ease of implementation, and strong search capabilities. SOA has been successfully employed in photovoltaic solar energy systems [30], power transformer fault diagnosis [31], collision-aware routing [32], and short-term wind speed forecasting [33].

SOA is based on the way seagulls naturally move and make their attacks on migratory birds as their victims cross the sea. The predation process of seagulls consists of migration and attacking behaviors. The migration behaviors of seagulls are corresponded to global search because of a large-scale flight. The attack behaviors of seagulls correspond to local search since seagull attacks their prey in a spiral fashion. Migration behavior is concerning the movement of seagulls from one place to another for finding the most abundant food sources. During migration, seagulls will judge the relative direction of themselves and the current elite seagulls while maintaining a distance from other individuals to avoid collision between seagulls. When the seagull successfully judges the relative direction of the elite seagull, it begins to determine the relative distance to the elite seagull. Seagulls use their wings and weight to keep altitude during migration. Attacking behavior is that seagulls frequently attack migrating birds over the sea when they migrate from one place to another. When finding prey, seagulls can continuously change the attack angle and flight speed to attack the prey.

2.3.2 Binary seagull optimization algorithm

Although the SOA is developed to solve continuous optimization problems, the decision space can be regarded as a hypercube in discrete optimization problems. The seagulls can move farther or closer to corners of the search space. Thus, the proposed BSOA utilizes a transfer function to generate the probability of updating position to '0' or '1' according to the speed of a seagull.

The BSOA has two algorithmic parameters, including a factor for controlling the movement behavior of the seagulls (A) and a factor for balancing between exploration and exploitation (B). Factor A is used to avoid the collision of seagulls during migration and affect the exploration of BSOA. A large value of factor A is used to strengthen exploration. When factor A is raised, the BSOA focuses primarily on exploration and intends to perform an extensive global search. The BSOA used a small factor B to strengthen exploitation. Factor B decides the influence of the position for the elite search agent. When the distance between the best position discovered so far and the individual position decreases, the perturbation

of the present position also reduces. The value of factor B is adaptively decreased when iteration is increased. Thus, the BSOA tends to perform a local search.

The notations used for BSOA are as follows. Ψ denotes the total amount of seagulls, $t_{\max}$ is the required iterations, $\mathbf{x}_i^t = [x_{i,1}^t, \dots, x_{i,K}^t]^T$ depicts the position of the i th seagull at iteration t , $\mathbf{z}_i^t = [z_{i,1}^t, \dots, z_{i,K}^t]^T$ depicts the search position of the i th seagull that does not collide with other seagulls at iteration t , $\mathbf{y}_i^t = [y_{i,1}^t, \dots, y_{i,K}^t]^T$ depicts the search direction of the i th seagull toward the elite seagull at iteration t , $\mathbf{D}_i^t = [D_{i,1}^t, \dots, D_{i,K}^t]^T$ depicts the distance between the i th seagull and the elite seagull at iteration t , $\mathbf{V}_i^t = [V_{i,1}^t, \dots, V_{i,K}^t]^T$ depicts the speed of the i th seagull at iteration t , and $\mathbf{x}^* = [x_1^*, \dots, x_K^*]^T$ depicts the position of the best-so-far elite seagull. $A^t \in [A_{\min}, A_{\max}]$ denotes the factor A at iteration t , where $A_{\min}$ and $A_{\max}$ denote the lower and upper bound, respectively. $B^t \in [B_{\min}, B_{\max}]$ is the factor B at iteration t , where $B_{\min}$ and $B_{\max}$ denote the lower and upper bound, respectively.

The factors A and B are dynamically configured to enhance exploration in preceding searches and exploitation in subsequent searches. The variation of A is decreased exponentially. A large A is desirable to favor exploration at the beginning, and then, a small A helps the local search to focus on a promising region approaching the end. The variation of B is also exponentially decreased as iterations increase to perform a local search around the optimum.

To summarize, the BSOA is as follows. **Algorithm I: The BSOA**

Step 1: Parameter setting

Configure the parameters of Ψ , $A_{\min}$, $A_{\max}$, $B_{\min}$, $B_{\max}$, $t_{\max}$, and let $t = 0$, where t is the index of iteration.

Step 2: Initialization

- (a) Initialize a population of Ψ feasible seagulls with positions $\mathbf{x}_i^0$.

$$x_i^0 = \text{binran}[0, 1], i = 1, \dots, \Psi \quad (10)$$

where $\text{binran}[0, 1][0, 1]$ generates a binary random number (0 or 1), $x_{i,k}^0 \in \{0, 1\}$, $i = 1, \dots, \Psi$, $k = 1, \dots, K$.

- (b) Compute the fitness $f(\mathbf{x}_i^0)$ of a seagull collocated with RMTB, $i = 1, \dots, \Psi$.

Step 3: Adjust the two control factors

$$A^t = A_{\min} + (A_{\max} - A_{\min}) \times \exp\left(\ln\left(\frac{A_{\min}}{A_{\max}}\right) \times \frac{t}{t_{\max}}\right) \quad (11)$$

$$B^t = B_{\min} + (B_{\max} - B_{\min}) \times \left(1 - \exp\left(\frac{B_{\max}}{B_{\min}} \times \left(\frac{t}{t_{\max}} - 1\right)\right)\right) \quad (12)$$

Step 4: Collision avoidance.

Configure the search position of a seagull to avoid collisions between neighboring seagulls.

$$z_i^t = A^t \times x_i^t \quad i = 1, \dots, \Psi \quad (13)$$

Step 5: Movement toward the elite seagull.

- (1) Sort the Ψ seagulls according to their fitness from the biggest to the least, then select the elite seagull $\mathbf{x}^*$.
- (2) Determine the search direction of a seagull toward the elite seagull.

$$y_i^t = B^t \times (x_i^* - x_i^t) \quad i = 1, \dots, \Psi \quad (14)$$

- (3) Calculate the Manhattan distance between a seagull and the elite seagull.

$$D_i^t = |z_i^t + y_i^t| \quad i = 1, \dots, \Psi \quad (15)$$

where $\|\cdot\|$ denotes the L1-norm.

Step 6: Change the attack angle and speed.

- (1) Determine the spiral movements rx , ry and rz in three planes (x, y, z) .

$$ra = \nu \times e^{w \times \text{rand}[0, 2\pi]} \quad (16)$$

$$rx = ra \times \cos(\text{rand}[0, 2\pi]) \quad (17)$$

$$ry = ra \times \sin(\text{rand}[0, 2\pi]) \quad (18)$$

$$rz = ra \times \text{rand}[0, 2\pi] \quad (19)$$

where ra denotes the radius of a turn in a spiral, $\text{rand}[0, 2\pi]$ is the angle of attack, which is a random number in the range between 0 and 2π , and ν and w are constants to design the shape of the spiral.

- (2) Determine the speed of a seagull.

$$V_i^{t+1} = rx \times ry \times rz \times D_i^t \quad i = 1, \dots, \Psi \quad (20)$$

Step 7: Update the position of each seagull

- (1) Apply a V-shaped transfer function to convert the velocity of a seagull into a probability.

$$PR(V_{i,k}^{t+1}) = \left| \tanh(V_{i,k}^{t+1}) \right| \quad i = 1, \dots, \Psi \quad k = 1, \dots, K \quad (21)$$

- (2) Convert the continuous value into a binary value.

$$x_{i,k}^{t+1} = \begin{cases} 1 - x_{i,k}^t, & \text{if } PR(V_{i,k}^{t+1}) > \text{rand}[0, 1] \\ x_{i,k}^t, & \text{otherwise} \end{cases} \quad i = 1, \dots, \Psi \quad k = 1, \dots, K \quad (22)$$

Step 8: Update elitism

Calculate the fitness $F(\mathbf{x}_i^{t+1})$ and $F(\mathbf{x}^*)$ collocated with RMTB, then adopt the greedy method between $\mathbf{x}_i^{t+1}$ and $\mathbf{x}^*$. If $F(\mathbf{x}_i^{t+1}) > F(\mathbf{x}^*)$, set $\mathbf{x}^* = \mathbf{x}_i^{t+1}$, $i = 1, \dots, \Psi$.

Step 9: Stop criteria.

If $t \geq t_{\max}$, stop; else, let $t = t + 1$ and go to Step 3.

The BSOA terminates once the required amount of iterations $t_{\max}$ have been performed. Once the BSOA is completed, the final Ψ seagulls are ranked according to their fitness from the biggest to the least. After that, the former N seagulls are collected to construct a candidate subset.

2.3.3 An illustrative example

After describing Algorithm I, a simple knapsack example is assumed to improve the readability of the proposed BSOA. Consider the following knapsack problem. Suppose that there are three potential items: 1, 2, and 3, in addition to giving their weights and profits in Table 1. The goal is to find the optimal number of items $\mathbf{x} = [x_1, x_2, x_3]^T$ that will maximize the total profit $f(\mathbf{x})$ while preserving the capacity of the knapsack.

$$\max f(x) = \sum_{k=1}^3 \alpha_k \cdot x_k \quad (23)$$

$$\text{subject to } h(x) = \sum_{k=1}^3 \beta_k \cdot x_k < 15 \quad (24)$$

$$x_k \in \{0, 1\}, k = 1, \dots, 3 \quad (25)$$

where α_k denotes the profit, β_k denotes the weight, and $x_k=0$ indicates that the item in this position is not selected and 1 indicates selected. The constrained problem is transformed into an unconstrained one through penalty function methods.

$$\max F(x) = f(x) - pe(x) \quad (26)$$

Table 1 Profits and weights of three items

Item k	Profit α_k	Weight β_k
1	5	2
2	10	30
3	15	10

where $F(\mathbf{x})$ indicates a penalized objective value, and $pe(\mathbf{x})$ denotes the following quadratic penalty function.

$$pe(x) = \begin{cases} 0, & \text{if } h(x) < 15, \\ (h(x) - 15)^2, & \text{else.} \end{cases} \quad (27)$$

At the start, the proposed BSOA creates a population consisting of 4 seagulls and each seagull includes 3 dimensions. After creating and initializing the population, total profit, capacity, and penalized objective value of each seagull are computed. Table 2 shows the results of the initial population. The elite seagull of the initial population is $\mathbf{x}^* = [0, 0, 1]^T$ with total profit 15. Table 3 shows the migration process for iteration 1. Firstly, the two control factors A^0 and B^0 are adjusted using Eq. (11) and (12), respectively. During the migration of iteration 1, group of seagulls is moved from one place to another. A seagull must satisfy the following three conditions: collision avoidance, movement toward elite seagull, and closeness toward elite seagull. The speed of a seagull $\mathbf{V}_i^1$ is updated according to the spiral movement and position of the elite seagull using Eq. (20). The new position of each seagull is updated using Eq. (22). Table 4 shows the results of migration for iteration 1. The elite seagull of iteration 1 is $\mathbf{x}^* = [1, 0, 1]^T$ with total profit 20.

Table 2 Results of the initial population

i	$\mathbf{x}_i^0$	$f(\mathbf{x}_i^0)$	$h(\mathbf{x}_i^0)$	$F(\mathbf{x}_i^0)$	$\mathbf{x}^*$
1	$[0, 1, 1]^T$	25	40	-600	$[0, 0, 1]^T$
2	$[0, 0, 1]^T$	15	10	15	
3	$[1, 0, 0]^T$	5	2	5	
4	$[1, 1, 1]^T$	30	42	-714	

Table 3 Migration process for iteration 1

i	A^0	B^0	$\mathbf{z}_i^0$	$\mathbf{y}_i^0$	$\mathbf{D}_i^0$	$\mathbf{V}_i^1$	$\mathbf{x}_i^1$
1	2	4	$[0, 2, 2]^T$	$[0, -4, 0]^T$	$[0, -2, 2]^T$	$[0, -1, 1]^T$	$[0, 1, 0]^T$
2	2	4	$[0, 0, 2]^T$	$[0, 0, 0]^T$	$[0, 0, 0]^T$	$[0, 0, 0]^T$	$[0, 0, 1]^T$
3	2	4	$[2, 0, 0]^T$	$[-4, 0, 4]^T$	$[-2, 0, 4]^T$	$[-1, 0, 2]^T$	$[1, 0, 1]^T$
4	2	4	$[2, 2, 2]^T$	$[-4, -4, 0]^T$	$[-2, -2, 2]^T$	$[-1, -1, 1]^T$	$[1, 1, 0]^T$

Table 4 Results of the migration for iteration 1

i	$\mathbf{x}_i^1$	$f(\mathbf{x}_i^1)$	$h(\mathbf{x}_i^1)$	$F(\mathbf{x}_i^1)$	$\mathbf{x}^*$
1	$[0, 1, 0]^T$	10	30	-215	$[1, 0, 1]^T$
2	$[0, 0, 1]^T$	15	10	15	
3	$[1, 0, 1]^T$	20	12	20	
4	$[1, 1, 0]^T$	15	32	-274	

2.4 Exploitation

Finally, the ROCBA is proceeded to find an illustrious solution from the N exceptional solutions. The ROCBA provides an alternative of asymptotically optimal allocation of computational effort among solutions in the exploration. In the ROCBA, less computing budget is spent on non-critical solutions, and more computing budget is spent on critical solutions. Focusing on a few critical solutions not only reduces computing effort but also decreases the critical solutions' variances. ROCBA is utilized to raise the efficiency of OO by allocating the computing budget reasonably. OO theory distributes equal computation to each solution, which will hardly reach the highest probability of correct selection using a specific computing budget. ROCBA is proposed to enhance the probability of correct selection by allocating computing budget to each solution according to their performances. Thus, ROCBA can provide a recipe for asymptotically optimal allocation of computing budget.

Through the statistics resulting from the N exceptional solutions, the quantity of extra replications is provided for exceptional solutions. Let C_b represents the allowable computing effort, L_0 indicates the basic replications for an exceptional solution, and L_i denotes the replications allocated to the i th exceptional solution. An additional computing budget, Δ , is added at each iteration to increase the probability of correct selection. The value of Δ is typically problem-dependent. A large value of Δ will waste of computing the budget to achieve an unnecessary high confidence level, while a small value of Δ will lead to performing the budget allocation procedure many times. The target of the ROCBA is to intelligently allocate C_b to $L_1, L_2, \dots, L_N$ that maximizes the probability of choosing the optimum as well as satisfies $L_1 + L_2 + \dots + L_N = C_b$. The allowable computing effort C_b is computed by $C_b = \frac{N \times L_a}{s}$, where L_a represents the replications used in the accurate estimate, and s denotes a time-reducing parameter corresponding to N which is referred to the OCBA analysis [34, 35].

To enhance the computing efficiency of the primitive OCBA, the ROCBA is proposed in this work. In the primitive OCBA, all replications must carry out repeatedly to compute the statistics of an exceptional solution. The ROCBA just needs to perform extra replications to obtain the statistics of an exceptional solution. The step-wise procedures of the ROCBA are as follows.

Algorithm II: The ROCBA

Step 1. Configure the parameter of s , L_0 , and L_a . Set $l = 0$, $L_1^l = L_0, \dots, L_N^l = L_0$ and determine the value of $C_b = \frac{N \times L_a}{s}$.

Step 2. Add an additional computing effort Δ to $\sum_{i=1}^N L_i^l$, and update the replications of an exceptional solution.

$$L_j^{l+1} = \left(\sum_{i=1}^N L_i^l + \Delta \right) \times \theta_j^l / \left(\theta_b^l + \sum_{i=1, i \neq b}^N \theta_i^l \right) \quad (28)$$

$$L_b^{l+1} = \frac{\theta_b^l}{\theta_j^l} \times L_j^{l+1} \quad (29)$$

$$L_i^{l+1} = \frac{\theta_i^l}{\theta_j^l} \times L_j^{l+1} \quad (30)$$

where $\frac{\theta_i^l}{\theta_j^l} = \left(\frac{\delta_i^l \times (\bar{f}_b^l - \bar{f}_j^l)}{\delta_j^l \times (\bar{f}_b^l - \bar{f}_i^l)} \right)^2$, $\theta_b^l = \delta_b^l \sqrt{\sum_{i=1, i \neq b}^N (\frac{\theta_i^l}{\delta_i^l})^2}$, $\bar{f}_i^l = \frac{1}{L_i^l} \sum_{h=1}^{L_i^l} f_h(x_i)$ for all $i \neq j \neq b$, $b = \arg \max_i \bar{f}_i^l$, $\mathbf{x}_i$ depicts the i th exceptional solution, and $f_h(\mathbf{x}_i)$ depicts the objective value of $\mathbf{x}_i$ at the h th replication.

Step 3. Carry out extra replications with $\max [0, L_i^{l+1} - L_i^l]$ to the i th exceptional solution, and compute the extra mean ($\hat{f}_i^{l+1}$) and extra standard deviation ($\hat{\delta}_i^{l+1}$).

$$\hat{f}_i^{l+1} = \frac{1}{(L_i^{l+1} - L_i^l)} \sum_{h=L_i^l+1}^{L_i^{l+1}} f_h(x_i) \quad (31)$$

$$\hat{\delta}_i^{l+1} = \sqrt{\frac{1}{(L_i^{l+1} - L_i^l)} \sum_{h=L_i^l+1}^{L_i^{l+1}} (f_h(x_i) - \hat{f}_i^{l+1})^2} \quad (32)$$

Step 4. Calculate the new mean ($\bar{f}_i^{l+1}$) and new standard deviation (δ_i^{l+1}) of the i th exceptional solution for overall replications.

$$\bar{f}_i^{l+1} = \frac{1}{L_i^{l+1}} \left(L_i^l \times \bar{f}_i^l + (L_i^{l+1} - L_i^l) \times \hat{f}_i^{l+1} \right) \quad (33)$$

$$\delta_i^{l+1} = \sqrt{\frac{1}{(L_i^{l+1} - 1)} \times \left(L_i^l (\bar{f}_i^l)^2 + (L_i^l - 1) (\delta_i^l)^2 + (L_i^{l+1} - L_i^l) (\hat{f}_i^{l+1})^2 + (L_i^{l+1} - L_i^l - 1) (\hat{\delta}_i^{l+1})^2 - L_i^{l+1} (\hat{f}_i^{l+1})^2 \right)} \quad (34)$$

Step 5. If $\sum_{i=1}^N L_i^l \geq C_b$, terminate and pick the optimal $\mathbf{x}^*$ with the maximum objective value; else, set $l = l + 1$, return to Step 2.

2.5 The BSOO method

The flow chart of the BSOO method is shown in Fig. 2.

Algorithm III: The BSOO

Step 1: Configure the quantities of Ψ , $A_{\min}$, $A_{\max}$, $B_{\min}$, $B_{\max}$, $t_{\max}$, N , s , L_0 , L_a and Δ .

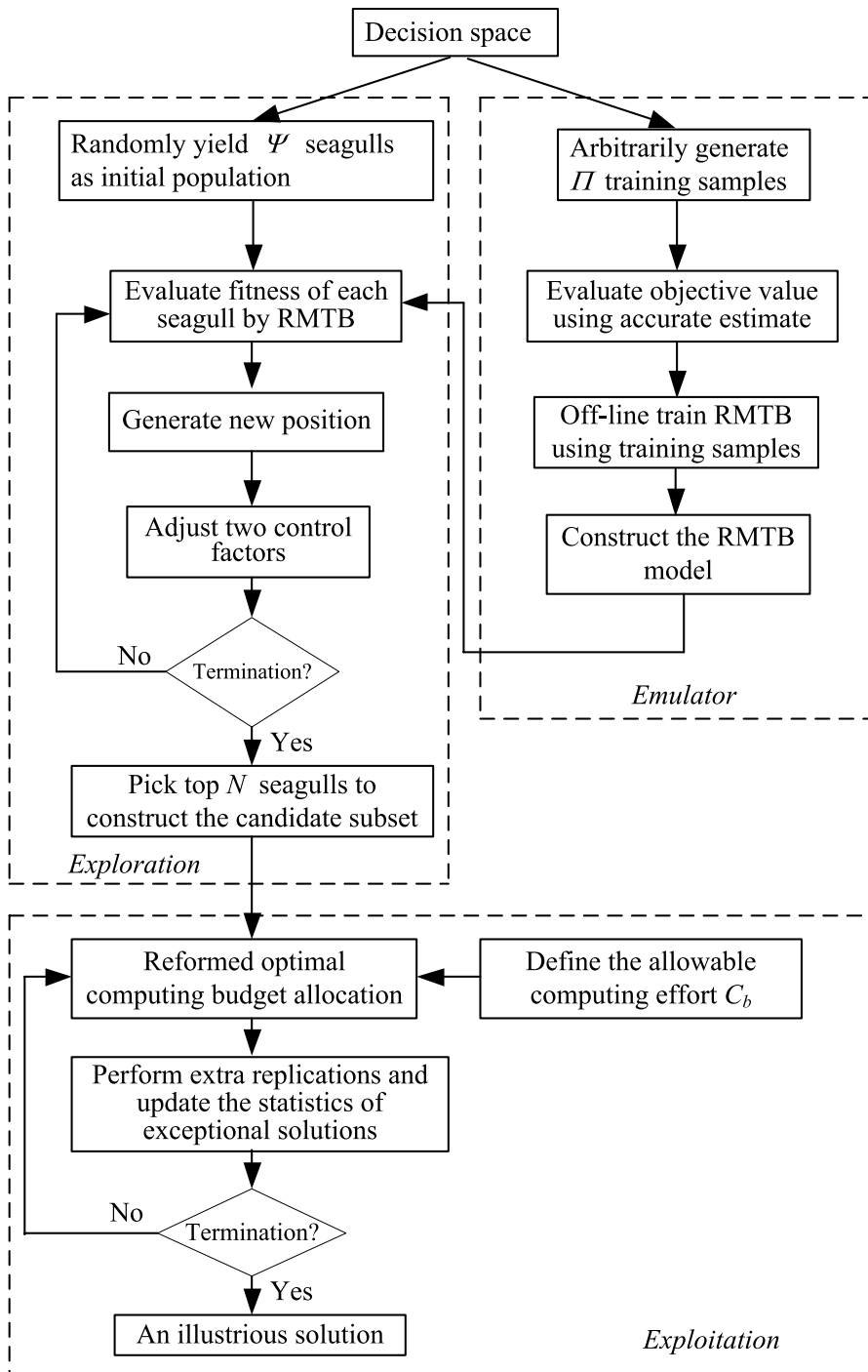


Fig. 2 Flow chart of the BSOO method

Step 2: Arbitrarily choose Π $\mathbf{x}$'s from the decision space and evaluate $F_a(\mathbf{x})$ by accurate estimate. The RMTB is trained offline through these Π solutions.

Step 3: Produce Ψ feasible $\mathbf{x}$'s as the initial population, then adopt the BSOA to those individuals collocated with RMTB. After the BSOA terminates, rank all the final Ψ $\mathbf{x}$'s based on their approximate fitness from the highest to the lowest, and choose the prior N $\mathbf{x}$'s to construct a candidate subset.

Step 4: Adopt the ROCBA to the N exceptional solutions in the candidate subset and pick the optimum $\mathbf{x}^*$, which is the illustrious solution that we seek.

3 Optimal layout of shortcuts in the sorting conveyor system

3.1 Sorting conveyor system

A logistics distribution center typically receives hundreds or thousands of different items from suppliers every day. When the logistics distribution center receives an order, the sorting conveyor system is operated from the receiving zone to the storage zone. Figure 3 describes the operation of an automated system with the sorting process. The sorting conveyor system is composed of a sorting device, a sorting crossing, a conveying device, and a control device. The control device instructs the sorting device according to the sorting requirements, and can automatically sort the commodities according to the type of the commodities or the delivery location of the commodities, and decide which sorting gate a certain commodity should enter. Based on the sorting instruction provided by the control device, the sorting device makes the parcels change the transporting direction on the conveying device to enter the sorting gate or enter other conveyors. The conveying device moves the package through the control device and the sorting device. Two sides of conveying device are generally connected to several sorting crossings, and thus, the sorted packages can slide down the main conveyor for subsequent operations. The sorting crossing is the passage that removes the sorted parcels from the main conveyor and enters the collecting area.

Layout configuration of the sorting conveyor system is classified into five types based on the layout shape: single row, multi-row, multi-floor, loop layout, and open field. Among them, the loop layout is a layout where the machines are allocated in

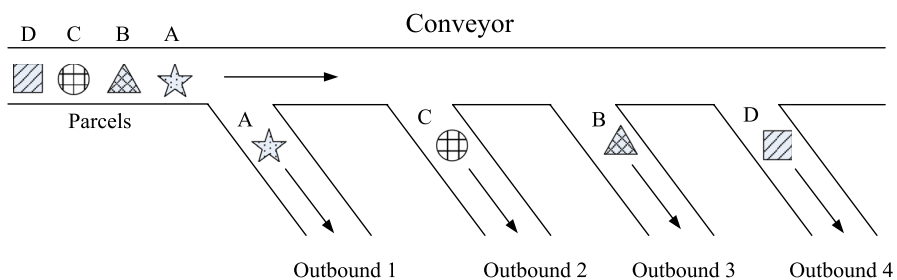


Fig. 3 Operation of an automated system with the sorting process

a close ring-like loop and the parcels are transported along this loop in one direction only. The loop sorting system consists of the inbound conveyors, outbound conveyors, shortcuts, and loop conveyors. Figure 4 shows a loop sorting conveyor system including shortcuts. There are two inbound conveyors which are arranged on two opposite sides of the loop conveyor. Various outbound conveyors represent different destinations of the parcels. Outbound conveyors are arranged evenly along with the sorting conveyor system, and the conveyors are transported in a counter-clockwise direction. The dashed line in Fig. 4 represents a shortcut. Shortcuts are additional paths in the loop sorting system to accelerate the process of transporting parcels. When a parcel reaches the loop conveyor, a shortcut can quickly transport the parcel to the opposite side of the loop. Since the shortcut shortens the length between two stations, both the capacity of the system and the utilization of the loop conveyor are increased due to reducing the occupied space. When we set the number of shortcuts in the system, we need to evaluate the facility cost, maintenance, and manpower. Accordingly, it is important to design the layout of shortcuts based on different conditions to reduce additional costs caused by changes.

In the past, when sorting conveyor systems faced the optimization of shortcut layout, they mostly used trial and error or the subjective experience of system designers as the basis for setting shortcut layouts. Operating costs and manpower allocation of picking personnel cause losses to the logistics distribution center. Subsequently, traditional optimization methods are gradually adopted to solve the optimal shortcut layout, including the exhaustive method, greedy algorithm, integer linear programming method, dynamic programming method, and branch-and-bound method. The

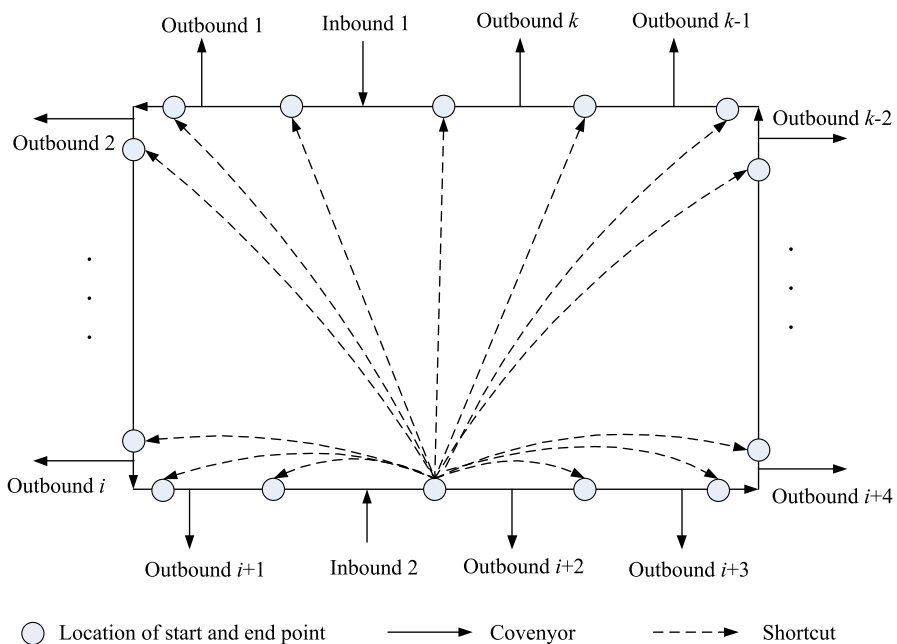


Fig. 4 A loop sorting conveyor system including shortcuts

exhaustive method is to test all the sample solutions one by one in order and find out the sample solutions that meet the requirements. It is a violent solution method with simple method ideas but high complexity. The greedy algorithm takes the optimal choice in the current state at each step [36] and expects to achieve a better result in the end, but cannot obtain the optimal solution in the whole domain. Integer linear programming uses complex mathematical expressions to construct patterns [37], but there are too many constraints. The dynamic programming method has a complicated solution process, which leads to a long computation time [38]. Although the branch-and-bound method can reduce the search solution space, when the worst case occurs, all the solution spaces need to be searched. When the problem is large, it will take too much time to solve it with traditional optimization methods. To avoid spending too much computation time, heuristic algorithms have been proposed recently to obtain near-optimal solutions [39]. However, heuristic algorithms still encounter the dilemma of being limited to local optimal solutions.

Fortunately, one possible alternative is to model the system as a CBSOP. Therefore, the optimal layout of shortcuts in the sorting conveyor system is firstly modeled as a CBSOP. Then, the proposed BSOO is adopted for finding the optimal layout of shortcuts for maximizing the throughput of the sorting conveyor system.

3.2 Mathematical Formulation

Listed below are the requirements and assumptions when formulating the CBSOP. (i) The allocation of each outbound conveyor is known. (ii) The destination of each parcel is known. (iii) The speed and length of the loop conveyor are known. (iv) The number of inbound conveyors, outbound conveyors, and parcel types is known. (v) The size and percentage of each parcel are given. (vi) The arrival interval rate of the parcel follows a given probability distribution. (vii) The scanning time of the zip code in a parcel is ignorable.

Now, the optimal layout of shortcuts in the sorting conveyor system is formulated as the following CBSOP.

$$\max E[f(X, \lambda)] \quad (35)$$

$$\text{subject to } \sum_{i=1}^I \sum_{j=1}^J x_{ij} = D, \forall i \neq j \quad (36)$$

$$x_{ij} \in \{0, 1\}, i = 1, \dots, I, j = 1, \dots, J \quad (37)$$

where I denotes the total amount of locations for start points of shortcuts, J is the total number of locations for endpoints of shortcuts, $\mathbf{X} = [x_{ij}]_{I \times J}$ denotes a solution matrix, x_{ij} is a 0–1 variable of the shortcut from start point i to endpoint j , $x_{ij}=1$ if a shortcut is set, $x_{ij}=0$ if there is no shortcut, D is the required number

of the shortcut, λ represents the arrival interval rate of the parcel, and $E[f(\mathbf{X}, \lambda)]$ is the throughput of the system, which is defined as the sum of the outputs for all out-bounds per hour.

The goal is to seek the optimal layout of shortcuts $\mathbf{X}^*$ to maximize the throughput $E[f(\mathbf{X}, \lambda)]$ for a given D . There is an alternative formula for estimating the value of $E[f(\mathbf{X}, \lambda)]$, which is the sample mean defined as follows.

$$\bar{f}(\mathbf{X}, \lambda) = \frac{1}{L} \sum_{\ell=1}^L f_{\ell}(\mathbf{X}, \lambda) \quad (38)$$

where L represents the number of replications, and $f_{\ell}(\mathbf{X}, \lambda)$ indicates the estimate of the ℓ th replication. The value of $\bar{f}(\mathbf{X}, \lambda)$ has a better estimate of $E[f(\mathbf{X}, \lambda)]$ when the quantity of replications increases. The value of L should be sufficiently large when a precise estimate about $E[f(\mathbf{X}, \lambda)]$ is needed. However, it is nearly impossible to evaluate the performance of every feasible solution when the number of shortcuts is increased. For example, if there are 20 connect points with two shortcuts, the number of available shortcuts is $2 \times C_2^{20} = 190$ and the number of feasible solutions is $C_2^{190} = 17955$. Since the constraints are soft, these constraints can be relaxed using penalty functions. Thus, the constrained minimization problem (35)-(36) can be transformed into an unconstrained one.

$$\max F(\mathbf{X}, \lambda) = \frac{1}{L} \sum_{\ell=1}^L f_{\ell}(\mathbf{X}, \lambda) - \eta \times \left(\sum_{i=1}^I \sum_{j=1}^J x_{ij} - D \right)^2 \quad (39)$$

where $\eta=10^6$ indicates a penalty weight, and $F(\mathbf{X}, \lambda)$ denotes a penalized objective value.

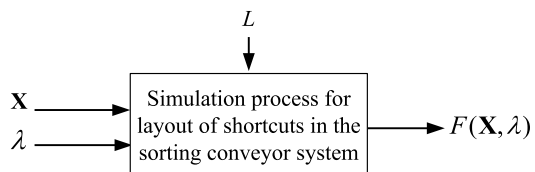
Figure 5 demonstrates the input/output relation for the layout of shortcuts in the sorting conveyor system, where $\mathbf{X}$ denotes a solution matrix, λ represents the arrival interval rate of the parcel, and $F(\mathbf{X}, \lambda)$ indicates the penalized objective value.

3.3 Apply the BSOO Method

3.3.1 Construct the emulator

There are four procedures to construct the RMTB emulator for evaluating a solution. (i) Arbitrary choose Π $\mathbf{X}$'s from decision space and compute $F_a(\mathbf{X}, \lambda)$ by accurate estimate, then represent those Π solutions and their estimations as $\mathbf{X}_i$ and

Fig. 5 The relation between input and output for the layout of shortcuts in the sorting conveyor system



$F_a(\mathbf{X}_i, \lambda)$, respectively. (ii) Determine regularization coefficients and choose suitable parameters of cubic Hermite splines. (iii) Calculate the discretized matrix of the continuous energy function. (iv) Compute the coefficients of the multivariate B-splines.

3.3.2 Determine the candidate subset

With the aid of the RMTB emulator, N exceptional solutions are determined by the BSOA. Firstly, Ψ feasible seagulls are arbitrarily produced to be the initial population. The objective value of a seagull is obtained by the RMTB emulator. When the BSOA terminated after $t_{\max}$ iterations, the Ψ seagulls are sorted based on their fitness from the highest to the lowest. The former N seagulls are chosen to construct the candidate subset.

3.3.3 Determine the illustrious solution

Finally, the ROCBA is utilized to find an illustrious solution from the N exceptional solutions. The setting of N cannot be too large to save computational time. Nevertheless, some excellent solutions will lose if the value of N is too small. Chen et al. [33] provided that a proper value of L_0 is in the range between 5 and 20, and a good setting of Δ is larger than 10% of N but smaller than 100.

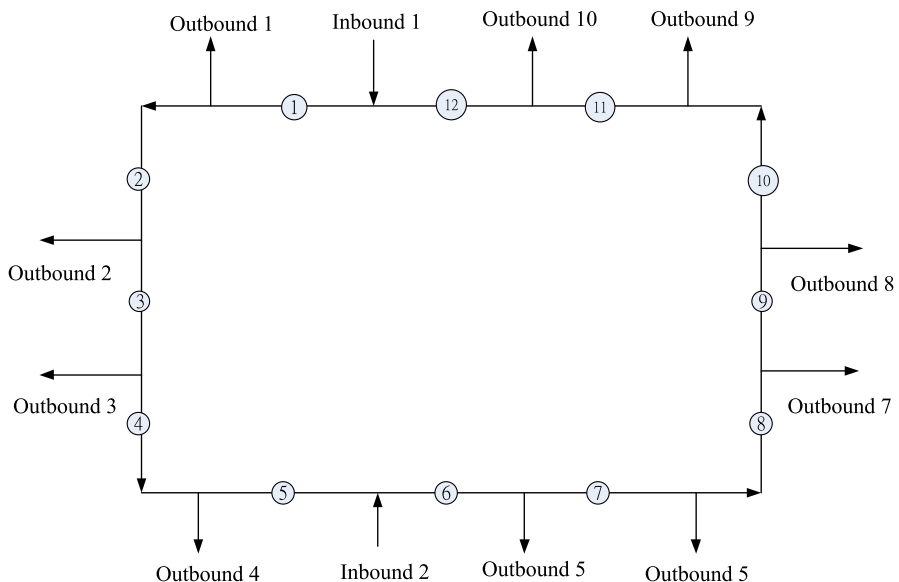


Fig. 6 Layout of loop sortation system

4 Experimental results and comparison

4.1 Experimental example

A practical conveyor sortation system presented in [39] is utilized to test the BSOO method. Figure 6 illustrates the layout of the loop sortation system, which has 2 inbound conveyors, 10 outbound conveyors, 12 nodes, and a loop conveyor. The 2 inbound conveyors are arranged on two opposite sides of the loop conveyor. The 10 outbound conveyors represent different destinations of the parcels. The 12 nodes represent the locations of the start point and endpoint. Outbound conveyors are arranged evenly along with the sorting conveyor system, and the conveyors are transported in a counterclockwise direction. When a parcel reached the system, it was loaded onto one inbound conveyor. Then, this parcel was transported to one of the outbound conveyors based on its destination. An outbound conveyor only served a type of parcel. The goal is to maximize the throughput of the system, which is defined as the sum of the outputs for all outbounds per hour. When a parcel meets a shortcut crossroads, three rules are applied as below. (i) We choose the shortcut with an endpoint before the parcel's destination. (ii) If available shortcuts can be transferred to another shortcut, we choose the shortcut which is opposed to staying on the loop conveyor. (iii) If the available shortcuts are more than one, we choose the shortcut with an endpoint that is close to the destination of the parcel number.

Table 5 shows the information on the ten types of parcel and the corresponding outbound conveyor. The arrival interval rate of the parcel λ follows a Poisson probability distribution with a mean of 0.1 (1/s). Thus, at least a parcel will arrive within 10 s. A parcel was placed into a container whose size was fixed at 0.5 m. The frequency of occurrence of every parcel type was fixed at 0.1. The frequency of occurrence is defined as a parcel type reached the system divided by all parcel types. Since the frequency of occurrence of every parcel type was the same, the number of parcels of every outbound conveyor could be balanced. Each conveyor traveled at a fixed speed of 2 m/s, and thus, the throughput of the shortcut was

Table 5 Information on the ten types of parcel

Type	Outbound conveyor	Parcel length (m)	Frequency of occurrence
(i)	1	0.5	0.1
(ii)	2	0.5	0.1
(iii)	3	0.5	0.1
(iv)	4	0.5	0.1
(v)	5	0.5	0.1
(vi)	6	0.5	0.1
(vii)	7	0.5	0.1
(viii)	8	0.5	0.1
(ix)	9	0.5	0.1
(x)	10	0.5	0.1

2 m/s. Accordingly, the capacity with the loop sortation system without shortcuts is 19113, and the total number of parcels per hour is 28800. Since the experiment was constructed to simulate the operation and the throughput of the sortation system with predefined assumptions, there is no existing dataset used in this research.

4.2 Experimental Results

Five cases with various numbers of shortcuts are adopted to test the BSOO method. Cases I to V have two, four, six, eight, and ten shortcuts, respectively. There are 16,641 randomly chosen samples to train the RMTB. The value $\Pi=16,641$ was computed using the sample size calculator with a confidence interval of 1% and a confidence level of 99% [40]. Regarding the parameters for modeling RMTB, $\delta=0.5$ and $\xi=10^{-14}$ were determined by the analytic approach. Due to the random nature, 30 hand-tuning experiments were performed to configure the parameters. The parameters adopted in BSOA were $A_{\max}=2$, $A_{\min}=0.1$, $B_{\max}=4$, $B_{\min}=0.5$, $\nu=1$, $w=1$, $t_{\max}=200$, $\Psi=30$, $\Psi=40$, $\Psi=50$, $\Psi=70$, and $\Psi=100$ for cases I to V, respectively. Numerous hand-tuning experiments show that BSOA using the above values performed better over 30 trials. Figure 7 demonstrates the variations of two factors A and B over 200 iterations. The BSOA efficiently explored the decision space in former iterations while exploiting the identified search region in later iterations. The setting of exceptional solutions was $N=20$. The parameters employed in ROCBA were $L_0=20$, $\Delta=10$ and $L_a=10^4$. The time-reducing parameter s with respect to $N=20$ is 8.2 [34]. Therefore, the allowable computing effort C_b was 24,390. RMTB was available under an open-source license as part of the surrogate modeling toolbox (SMT) in Python. The simulation process for the layout of shortcuts in the sorting conveyor system was implemented in MATLAB/Simulink and Stateflow. The proposed BSOO approach was implemented on MATLAB R2018a

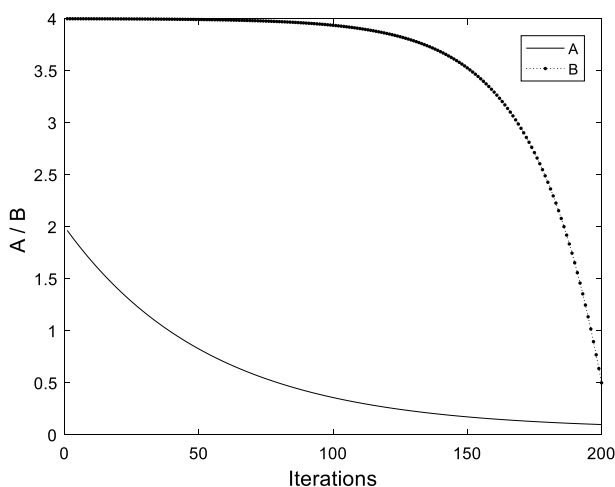


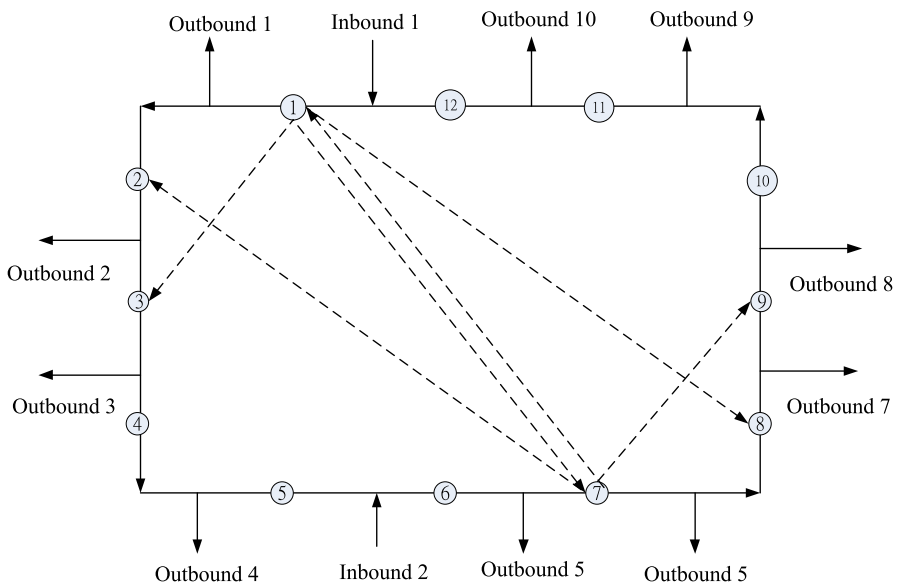
Fig. 7 Variations of A and B over iterations

Table 6 The illustrious solution $\mathbf{X}^*$, throughput, and CPU times of five cases

Case	$\mathbf{X}^*$	Throughput	CPU time (sec.)
I (2 SHORTCUTS)	$1 \rightarrow 8, 7 \rightarrow 2$	26,081.22	32.65
II (4 SHORTCUTS)	$1 \rightarrow 7, 1 \rightarrow 8, 7 \rightarrow 1, 7 \rightarrow 2$	27,553.46	46.51
III (6 SHORTCUTS)	$1 \rightarrow 3, 1 \rightarrow 7, 1 \rightarrow 8, 7 \rightarrow 1, 72, 7 \rightarrow 9$	28,438.62	57.43
IV (8 SHORTCUTS)	$1 \rightarrow 3, 1 \rightarrow 8, 4 \rightarrow 7, 7 \rightarrow 9, 710, 8 \rightarrow 1, 10 \rightarrow 1, 10 \rightarrow 2$	28,420.78	80.77
V (10 SHORTCUTS)	$1 \rightarrow 3, 14, 4 \rightarrow 7, 5 \rightarrow 7, 5 \rightarrow 8, 7 \rightarrow 2, 7 \rightarrow 9, 81, 10 \rightarrow 1, 11 \rightarrow 1$	28,416.54	116.32

software, which was executed on a personal computer with Intel Core i7-7500U 2.70 GHz, 32 GB RAM, and Windows 10 as an operating system.

Table 6 demonstrates the illustrious solution $\mathbf{X}^*$, throughput, and CPU times of five cases of the first trial. The obtained illustrious solution $\mathbf{X}^*$ is a matrix with the form $[x_{ij}^*]_{12 \times 12}$. For case I, $x_{ij}^* = \begin{cases} 1, i = 1, j = 8, \text{ and } i = 7, j = 2 \\ 0, \text{ otherwise} \end{cases}$. The symbol “ $i \rightarrow j$ ” is used to improve readability from matrix form in the obtained illustrious solution, which means that the shortcut is from the start point i to the endpoint j . Test result reveals that case III is the most applicable solution for the test example. Figure 8 displays the layout of loop sortation system for case III. The consumed CPU time of case III is less than one minute, which shows that the BSOO can be adopted in a real-time application. Because redundant shortcuts may cause a diseconomy layout, the throughputs do not significantly improve with further

**Fig. 8** Layout of loop sortation system for case III

increases for cases IV and V. Since the RMTB emulator is trained offline, its training time is not included in the CPU time consumed by the BSOO. Thus, the CPU time consumed by the BSOO is the sum of the execution time of the BSOA for exploration and the ROCBA for exploitation.

4.3 Comparison

The BSOO method has been compared to five heuristic approaches, GA, ACO, CSO, binary whale optimization algorithm (bWOA), and binary equilibrium optimization (BEO) for case III. In the GA [6], a population size of 50, single-point crossover with a probability of 0.8, roulette wheel selection, and uniform mutation with a rate of 0.01 were employed. In the ACO [7], a population size of 50, the global pheromone volatile factor with 0.3, the local pheromone evaporation rate with 0.5, the control factor between the relative proportion of the exploitation and biased exploration with 0.9, the parameter to control the relative importance of information with 1, and initial pheromone of 0.1 were adopted. In the CSA [8], a population size of 50, receptor editing rate of 0.05, and the parameter control strength of mutation of 10 were applied. In the bWOA [13], a population size of 50, the shape of a logarithmic spiral with 2, and the V-shaped transfer functions were utilized. In the BEO [14], a population of particles with 50, the control factor of diversification with 3, the control factor of exploitation with 1, the factor of generation rate of 0.5, and the V-shaped transfer functions were utilized.

The accurate estimate was utilized to evaluate the objective value for five heuristic approaches. Due to the random nature, 30 trials were performed to test and verify the performance of six approaches. Since five heuristic approaches need a long searching process to yield the optimum, the searching processes ceased when they had spent 30 min of computational time. Table 7 presents that the means of the best-so-far objective values resulted from GA, ACO, CSA, bWOA, and BEO which were 8.36%, 13.73%, 11.57%, 15.04%, and 10.04% smaller than that resulted from BSOO, respectively. Simulation results demonstrate that the BSOO outperforms five heuristic approaches even after the latter took more than 30 times the CPU time that was consumed by BSOO upon completion. Since the fitness in each of the five heuristic methods was evaluated using an accurate estimate, their best-so-far objective values improved slowly. Problem [35–37] is difficult to solve because (i) evaluation of the throughput is time-consuming, and (ii) the discrete search space is huge. Actually, OO is not a method on its own but rather a supplement to BSOA. Therefore, BSOA could be incorporated into OO to resolve issues (i) and (ii) simultaneously. The five heuristic methods associated with the accurate estimate are likely to outperform the proposed BSOO given a long enough execution time.

Furthermore, the analysis concerning the orders of obtained solutions using four methods was conducted. Because deciding the orders of all feasible solutions is impossible, a representative subset, Ω , is constructed to approximately represent the characteristics of the large decision space. Accordingly, 66,564 samples were arbitrarily chosen from the decision space to construct the representative subset and whose corresponding objective values were evaluated by accurate estimate. The

Table 7 Statistics of six approaches over 30 trials

Approaches	Min	Max	MBOV [†]	$\frac{\text{§} - \text{MBOV}^*}{*} \times 100\%$	Standard Deviation	S.E.M	Average order percentage	Average CPU time(sec.)
BSOO	28,408.8	28,439.7	28,422.8	0	5.1	0.93	0.003%	57.5
GA with accurate estimate	25,956.5	26,133.2	26,046.6	8.36%	28.2	5.15	1.95%	1798
ACO with accurate estimate	24,404.5	24,660.7	24,520.3	13.73%	44.8	8.18	5.86%	1801
CSA with accurate estimate	25,022.2	25,285.6	25,134.3	11.57%	36.9	6.74	4.21%	1794
bWOA with accurate estimate	23,968.6	24,323.4	24,146.7	15.04%	51.3	9.37	6.27%	1796
BEO with accurate estimate	25,441.1	25,681.5	25,568.1	10.04%	33.7	6.15	3.78%	1791

[†]MBOV mean of the best-so-far objective value

§ *: MBOV resulted from BSOO

value of $|\Omega|=66,564$ was provided using the sample size calculation considering the finite population correction factor under a confidence interval of 0.5% and a confidence level of 99% [40].

Furthermore, an analysis of order percentage was performed to show the order of an illustrious solution in the representative subset. The order percentage of an illustrious solution is defined by $\frac{r}{|\Omega|} \times 100\%$, where r is the rank of an illustrious solution in Ω . Table 7 also shows the statistics concerning the means of best-so-far objective values, average order percentages, and average CPU times over 30 trials resulting from six approaches. The standard error of the mean (S.E.M.) for the best-so-far objective value that was obtained by BSOO over 30 trials was 0.91. This small value indicates that most of the illustrious solutions obtained by the BSOO are quite close to the near-optimum in 30 trials.

5 Conclusion

To resolve the CBSOP in a short computing time, an approach incorporating BSOA into OO was presented. The BSOO comprises three modules including emulator, exploration, and exploitation. The RMTB emulator was very effective in quickly evaluating a solution. The BSOO utilized the BSOA for exploration with the ROCBA for exploitation. The BSOO was adopted for the optimal layout of shortcuts in the sorting conveyor system, which was modeled as a CBSOP. A practical conveyor sortation system with a loop conveyor was adopted as a test example. Five cases with various numbers of shortcuts were used to test the proposed BSOO. Test results revealed that case III was the most applicable solution for the test example. The consumed CPU time of case III was less than one minute, which shows that the BSOO can be adopted in a real-time application. The BSOO was compared with five heuristic approaches—GA, ACO, CSA, bWOA, and BEO collocated with an accurate estimate. The means of the best-so-far objective values resulted from GA, ACO, CSA, bWOA, and BEO which were 8.36%, 13.73%, 11.57%, 15.04%, and 10.04% smaller than that resulted from BSOO, respectively. Experimental results presented that most of the illustrious solutions resulting from the BSOO are quite close to the near-optimum in 30 trials. Since most of the previous works did not consider shortcuts in a sortation system, the paper's contribution is twofold: academic and practical perspectives. From the academic perspective, the BSOO is proposed to determine the optimal shortcut design of the sortation system. From the practical perspective, this paper utilizes a practical case with 2 inbound conveyors, 10 outbound conveyors and a loop conveyor to validate the applicability of the proposed approach. Future studies will concentrate on the application of OO to solve stochastic optimization problems with dominance constraints, including dominance-constrained portfolio problems and reinsurance-investment optimization problems.

Acknowledgements Shih-Cheng Horng is now a professor of the Department of Computer Science and Information Engineering at Chaoyang University of Technology, Taiwan, R.O.C. Shieh-Shing Lin is currently a professor of the Department of Electrical Engineering at St. John's University, Taiwan, R.O.C.

Author contributions All authors contributed to the study's conception and design. In addition, SC and SS performed material preparation, model building, and analysis. SC wrote the first draft of the manuscript. Finally, all authors read and approved the final manuscript.

Funding This research work is supported in part by the Ministry of Science and Technology in Taiwan, R.O.C., under Grant MOST111-2221-E-324-021.

Declarations

Conflict of interest The authors claim that they have no conflict of interest.

Ethical approval This study does not include any studies with human participants or animals performed by any of the authors.

References

1. Valiante E, Hernandez M, Barzegar A, Katzgraber HG (2021) Computational overhead of locality reduction in binary optimization problems. *Comput Phys Commun* 269:108102
2. Aslan M, Gunduz M, Kiran MS (2019) JayaX: Jaya algorithm with XOR operator for binary optimization. *Appl Soft Comput* 82:105576
3. Baykasoglu A, Ozsoydan FB, Senol ME (2020) Weighted superposition attraction algorithm for binary optimization problems. *Oper Res Int J* 20(4):2555–2581
4. Javaloyes-Anton J, Kronqvist J, Caballero JA (2022) Simulation-based optimization of distillation processes using an extended cutting plane algorithm. *Comput Chem Eng* 159:107655
5. Kobayashi K, Takano Y (2020) A branch-and-cut algorithm for solving mixed-integer semidefinite optimization problems. *Comput Optim Appl* 75(2):493–513
6. Anghelescu P (2021) Parallel optimization of program instructions using genetic algorithms. *CMC-Comput Mater Contin* 67(3):3293–3310
7. Sreelaja NK (2021) Ant colony optimization based light weight binary search for efficient signature matching to filter Ransomware. *Appl Soft Comput* 111:107635
8. Zhang WW, Zhang WZ, Yen GG, Jing HL (2019) A cluster-based clonal selection algorithm for optimization in dynamic environment. *Swarm Evolut Comput* 50:100454
9. Tang J, Liu G, Pan QT (2021) A review on representative swarm intelligence algorithms for solving optimization problems: applications and trends. *IEEE-CAA J Autom Sin* 8(10):1627–1643
10. Abualigah L, Yousri D, Abd Elaziz M, Ewees AA, Al-qaness MAA, Gandomi AH (2021) Aquila optimizer: a novel meta-heuristic optimization algorithm. *Comput Ind Eng* 157:107250
11. MirNaeimi F, Azizyan G, Rashki M (2021) Horse herd optimization algorithm: A nature-inspired algorithm for high-dimensional optimization problems. *Knowl-Based Syst* 213:106711
12. Mohammadi-Balani A, Nayeri MD, Azar A, Taghizadeh-Yazdi M (2021) Golden eagle optimizer: A nature-inspired metaheuristic algorithm. *Comput Ind Eng* 152:107050
13. Hussien AG, Hassanien AE, Houssein EH, Amin M, Azar AT (2020) New binary whale optimization algorithm for discrete optimization problems. *Eng Optim* 52(6):945–959
14. Abdel-Basset M, Mohamed R, Mirjalili S (2021) A binary equilibrium optimization algorithm for 0–1 knapsack problems. *Comput Ind Eng* 151:106946
15. Dhiman G, Kumar V (2019) Seagull optimization algorithm: Theory and its applications for large-scale industrial engineering problems. *Knowl-Based Syst* 165:169–196

16. Panagant N, Pholdee N, Bureerat S, Yildiz AR, Sait SM (2020) Seagull optimization algorithm for solving real-world design optimization problems. *Materials Testing* 62(6):640–644
17. Ho YC, Zhao QC, Jia QS (2007) Ordinal optimization: soft optimization for hard problems. Springer-Verlag, New York
18. Liu AD, Luh PB, Bragin MA, Yan B (2020) Ordinal-optimization concept enabled decomposition and coordination of mixed-integer linear programming problems. *IEEE Robot Autom Lett* 5(4):5051–5058
19. Horng SC, Lee CT (2021) Integration of ordinal optimization with ant lion optimization for solving the computationally expensive simulation optimization problems. *Appl Sci* 11(1):136
20. Horng SC, Lin SS (2020) Coupling elephant herding with ordinal optimization for solving the stochastic inequality constrained optimization problems. *Appl Sci* 10(6):2075
21. Horng SC, Lin SS (2018) Embedding ordinal optimization into tree-seed algorithm for solving the probabilistic constrained simulation optimization problems. *Appl Sci* 8(11):2153
22. Horng SC, Lin SS (2019) Bat algorithm assisted by ordinal optimization for solving discrete probabilistic bicriteria optimization problems. *Math Comput Simul* 166:346–364
23. Hwang JT, Martins JRRA (2018) A fast-prediction surrogate model for large datasets. *Aerosp Sci Technol* 75:74–87
24. Huang SH, Mahmud K, Chen CJ (2022) Meaningful trend in climate time series: A discussion based on linear and smoothing techniques for drought analysis in Taiwan. *Atmosphere* 13(3):444
25. Estrin R, Friedlander MP, Orban D, Saunders MA (2020) Implementing a smooth exact penalty function for equality-constrained nonlinear optimization. *SIAM J Sci Comput* 42(3):A1809–A1835
26. Arthur CK, Temeng VA, Ziggah YY (2020) Multivariate adaptive regression splines (MARS) approach to blast-induced ground vibration prediction. *Int J Min Reclam Environ* 34(3):198–222
27. Hu HD, Yu JY, Song YP, Chen F (2021) The application of support vector regression and mesh deformation technique in the optimization of transonic compressor design. *Aerosp Sci Technol* 112:106589
28. Chen Y, Tong CD, Ge YH, Lan T (2021) Fault detection based on auto-regressive extreme learning machine for nonlinear dynamic processes. *Appl Soft Comput* 106:107319
29. Han WX, Wang ZH, Shen Y, Xu B (2021) Interval estimation for uncertain systems via polynomial chaos expansions. *IEEE Trans Autom Control* 66(1):468–475
30. Subramanian A, Raman J (2022) Modified seagull optimization algorithm based MPPT for augmented performance of photovoltaic solar energy systems. *Automatika* 63(1):1–15
31. Wu YH, Sun XB, Zhang Y, Zhong XJ, Cheng L (2022) A power transformer fault diagnosis method-based hybrid improved seagull optimization algorithm and support vector machine. *IEEE Access* 10:17268–17286
32. Jagannathan P, Gurumoorthy S, Stateczny A, Divakarachar PB, Sengupta J (2021) Collision-aware routing using multi-objective seagull optimization algorithm for WSN-Based IoT. *Sensors* 21(24):8496
33. Chen X, Li YL, Zhang YC, Ye XL, Xiong X, Zhang FH (2021) A novel hybrid model based on an improved seagull optimization algorithm for short-term wind speed forecasting. *Processes* 9(2):387
34. Chen CH, Lee LH (2010) Stochastic simulation optimization: an optimal computing budget allocation. World Scientific, New Jersey
35. Xiao H, Gao F, Lee LH (2020) Optimal computing budget allocation for complete ranking with input uncertainty. *IIE Trans* 52(5):489–499
36. Chen K, Song MC, Zhang X, Wang SF (2016) Wind turbine layout optimization with multiple hub height wind turbines using greedy algorithm. *Renew Energy* 96:676–686
37. Fairclough H, Gilbert M (2020) Layout optimization of simplified trusses using mixed integer linear programming with runtime generation of constraints. *Struct Multidiscip Optim* 61(5):1977–1999
38. Yang X, He HB, Zhong XN (2021) Approximate dynamic programming for nonlinear-constrained optimizations. *IEEE Trans Cybernetics* 51(5):2419–2432
39. Chen TL, Chen JC, Huang CF, Chang PC (2021) Solving the layout design problem by simulation-optimization approach: a case study on a sortation conveyor system. *Simul Modelli Pract Theory* 106:102192
40. Ryan TP (2013) Sample size determination and power. John Wiley and Sons, New Jersey

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.

Authors and Affiliations

Shih-Cheng Horng¹ · Shieh-Shing Lin¹

Shieh-Shing Lin
sslin@mail.sju.edu.tw

¹ Department of Computer Science and Information Engineering, Chaoyang University of Technology, 168 Jifong E. Rd, Wufeng, Taichung 41349, Taiwan